

The `mcpServers` parameter configures external Model Context Protocol (MCP) servers that extend the agent's capabilities beyond built-in tools, enabling integration with third-party services and custom tooling. It accepts a JSON object where keys are server names and values contain server configuration including the required `command` to start the server, optional `args` array for command arguments, `env` object for environment variables, `timeout` in milliseconds (default: 120000), and `disabled` boolean flag.

Once configured, MCP server tools become available to the agent using the `@server_name/tool_name` syntax in the `tools` array, allowing agents to access external APIs, databases, or specialized services while maintaining the same permission and security model through `allowedTools` and `toolsSettings` parameters.

MCP Server Configuration Locations

- **Agent-Specific MCP Servers:**
 - Global agents: `~/.kiro/agents/*.json`
 - Workspace agents: `./.kiro/agents/*.json`
- **Legacy MCP Servers:**
 - Global: `~/.kiro/settings/mcp.json`
 - Workspace: `./.kiro/settings/mcp.json`

Control Legacy MCP Usage

```
{
  "useLegacyMcpJson": true
}
```

Default: true

How It Works

- When `useLegacyMcpJson: false`:
 - Only uses MCP servers defined in the agent config
- When `useLegacyMcpJson: true`:
 - Combines agent MCP servers + legacy MCP servers
 - **Priority:** Agent-specific > Workspace legacy > Global legacy
 - Agent-specific servers always win naming conflicts
 - **Examples**
 - Running any agent *in a workspace*:
 - Gets agent's own MCP servers
 - Gets workspace legacy MCP servers (`./.kiro/settings/mcp.json`)
 - Gets global legacy MCP servers (`~/.kiro/settings/mcp.json`)
 - Running any agent *outside workspace*:
 - Gets agent's own MCP servers
 - Gets global legacy MCP servers only

*Conflicts: If agent defines `git-tool` and legacy also defines `git-tool` → **agent version wins**, legacy skipped with warning*

Objective

Learn how to integrate external tools via Model Context Protocol (MCP) servers using `mcpServers` parameter.

Instructions

Step 1: Create Agent with AWS Knowledge MCP Server

```
1 cat > .kiro/agents/aws-knowledge-agent.json << 'EOF'
2 {
3   "name": "aws-knowledge-agent",
4   "description": "AWS documentation specialist",
5   "prompt": "You are an AWS documentation expert who helps developers find accurate, up-to-date information about AWS services. Don't use your own knowledge. If tools return error,",
6   "mcpServers": {
7     "aws-knowledge-mcp-server": {
8       "command": "uvx",
9       "args": [
10        "mcp-proxy",
11        "--transport",
12        "streamablehttp",
13        "https://knowledge-mcp.global.api.aws"
14      ],
15      "timeout": 30000
16    }
17  },
18  "tools": ["read", "@aws-knowledge-mcp-server"],
19  "allowedTools": ["read", "@aws-knowledge-mcp-server/aws__search_documentation"]
20 }
21 EOF
```

Step 2: Create Agent with AWS Pricing MCP Server

```
1 cat > .kiro/agents/aws-pricing-agent.json << 'EOF'
2 {
3   "name": "aws-pricing-agent",
4   "description": "AWS cost optimization specialist",
5   "prompt": "You are an AWS cost optimization expert who helps analyze pricing and recommend cost-effective solutions. Don't use your own knowledge. If tools return error,",
6   "mcpServers": {
7     "awslabs.aws-pricing-mcp-server": {
8       "command": "uvx",
9       "args": ["@awslabs.aws-pricing-mcp-server@latest"],
10      "env": {
11        "FASTMCP_LOG_LEVEL": "ERROR",
12        "AWS_REGION": "us-east-1"
13      },
14      "timeout": 30000
15    }
16  },
17  "tools": ["read", "@awslabs.aws-pricing-mcp-server"],
18  "allowedTools": ["read", "@awslabs.aws-pricing-mcp-server/get_pricing"]
19 }
20 EOF
```

Step 3: Test AWS Knowledge Integration

1. Start custome agent

```
1 kiro-cli chat --agent aws-knowledge-agent
```

2. Check mcp server

```
1 /mcp
```

Expected:

```
[aws-knowledge-agent] > /mcp
aws-knowledge-mcp-server
✓ aws-knowledge-mcp-server loaded in x.xx s
```

3. Check tools

```
1 /tools
```

Expected:

Tool	~Tokens	Permission
Built-in		
- read	870	trusted
aws-knowledge-mcp-server (MCP)		
- aws__read_documentation	1.5k	not trusted
- aws__recommend	450	not trusted
- aws__search_documentation	1.2k	trusted

4. Test these AWS knowledge commands:

```
"Search for information about AWS Lambda best practices"
"What are the key features of Amazon S3?"
"Find documentation about AWS VPC security groups"
"Explain AWS IAM roles and policies"
```

Step 4: Test AWS Pricing Integration

1. Start custome agent

```
1 kiro-cli chat --agent aws-pricing-agent
```

2. Check mcp server

```
1 /mcp
```

Expected:

```
[aws-pricing-agent] > /mcp
awslabs.aws-pricing-mcp-server
✓ awslabs.aws-pricing-mcp-server loaded in x.xx s
```

3. Check tools

```
1 /tools
```

Expected:

Tool	~Tokens	Permission
Built-in		
- read	870	trusted
awslabs.aws-pricing-mcp-server (MCP)		
- analyze_cdk_project	110	not trusted
- analyze_terraform_project	110	not trusted
- generate_cost_report	1.4k	not trusted
- get_bedrock_patterns	60	not trusted
- get_price_list_urls	350	not trusted
- get_pricing	3.8k	trusted
- get_pricing_attribute_values	400	not trusted
- get_pricing_service_attributes	250	not trusted
- get_pricing_service_codes	200	not trusted

4. Test these pricing commands:

```
"Get pricing information for EC2 t3.medium instances in ap-southeast-1"
"Compare pricing between different S3 storage classes"
"What's the cost of AWS Lambda for 1 million requests?"
"Show me RDS pricing for MySQL instances"
```

Step 5: Create Multi-MCP Server Agent

```
1 cat > .kiro/agents/aws-expert.json << 'EOF'
2 {
3   "name": "aws-expert",
4   "description": "Comprehensive AWS specialist with knowledge and pricing access",
5   "prompt": "You are a senior AWS solutions architect with access to comprehensive AWS documentation and pricing information. You design scalable, cost-effective, and secure",
6   "mcpServers": {
7     "aws-knowledge-mcp-server": {
8       "command": "uvx",
9       "args": [
10        "mcp-proxy",
11        "--transport",
12        "streamablehttp",
13        "https://knowledge-mcp.global.api.aws"
14      ],
15      "timeout": 30000
16    },
17     "awslabs.aws-pricing-mcp-server": {
18       "command": "uvx",
19       "args": ["@awslabs.aws-pricing-mcp-server@latest"],
20       "env": {
21         "FASTMCP_LOG_LEVEL": "ERROR",
22         "AWS_REGION": "us-east-1"
23       },
24       "timeout": 30000
25     }
26  },
27  "tools": [
28    "read",
29    "@aws-knowledge-mcp-server",
30    "@awslabs.aws-pricing-mcp-server"
31  ],
32  "allowedTools": [
33    "read",
34    "@aws-knowledge-mcp-server/aws__search_documentation",
35    "@awslabs.aws-pricing-mcp-server/get_pricing"
36  ]
37 }
38 EOF
```

Step 6: Test Multi-MCP Integration

1. Start custom agent

```
1 kiro-cli chat --agent aws-expert
```

2. Check mcp server

```
1 /mcp
```

Expected:

```
[aws-expert] > /mcp
awslabs.aws-pricing-mcp-server
✓ awslabs.aws-pricing-mcp-server loaded in 9.84 s

aws-knowledge-mcp-server
✓ aws-knowledge-mcp-server loaded in 10.50 s
```

3. Check tools

```
1 /tools
```

Expected:

Tool	~Tokens	Permission
Built-in		
- read	870	trusted
aws-knowledge-mcp-server (MCP)		
- aws__read_documentation	900	not trusted
- aws__recommend	420	not trusted
- aws__search_documentation	2.2k	trusted
awslabs.aws-pricing-mcp-server (MCP)		
- analyze_cdk_project	110	not trusted
- analyze_terraform_project	110	not trusted
- generate_cost_report	1.4k	not trusted
- get_bedrock_patterns	60	not trusted
- get_price_list_urls	350	not trusted
- get_pricing	3.8k	trusted
- get_pricing_attribute_values	400	not trusted
- get_pricing_service_attributes	250	not trusted
- get_pricing_service_codes	200	not trusted

4. Test combined functionality:

```
"Find AWS Lambda documentation and then get pricing for 1M requests"
"Research S3 best practices and compare storage class pricing"
"Design a cost-effective architecture for a web application"
```

Verification

- ☒ AWS Knowledge agent can search AWS documentation
- ☒ AWS Pricing agent can retrieve current pricing information
- ☒ Multi-MCP agent integrates both services seamlessly
- ☒ MCP tools appear with `@server_name/tool_name` format
- ☒ External tool integration works without errors

Key Learnings

- `mcpServers` parameter enables integration with external tools
- MCP server names become prefixes for tool names (`@server_name/tool_name`)
- Multiple MCP servers can be combined in a single agent
- Environment variables configure MCP server behavior
- Timeout settings prevent hanging operations
- MCP extends agent capabilities beyond built-in tools